

Cover page personal information redacted.

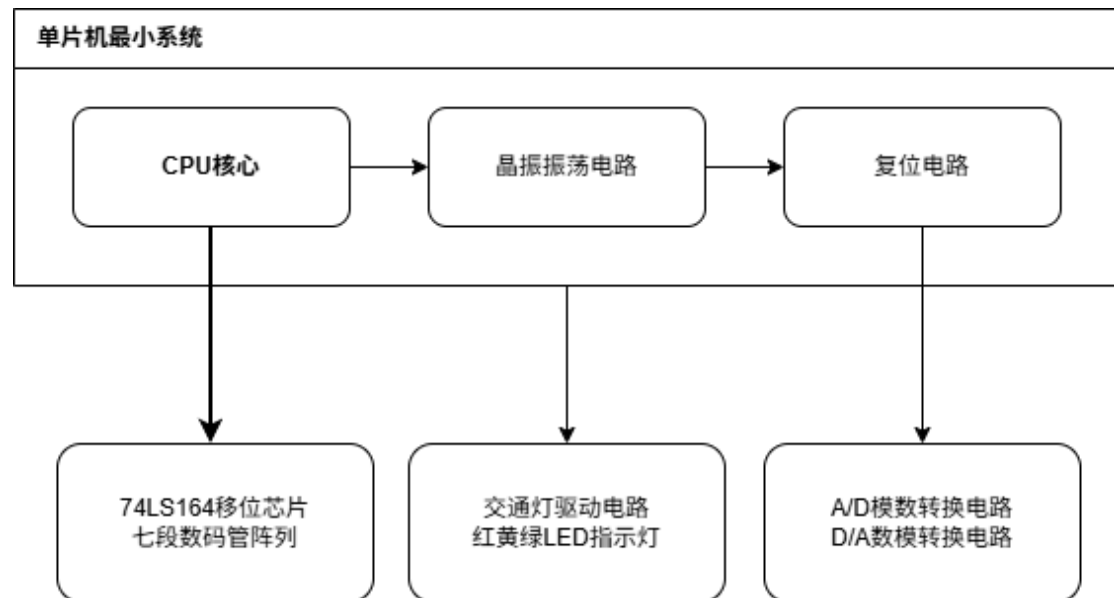
目录

一、设计环节一：单片机开发板的焊接与调试.....	1
1、单片机开发板的构成框图：	1
2、电路图：	1
3、仿真图：	2
4、焊接元件清单：	2
5、焊接后的开发板照片：	4
6、焊接过程中的注意事项：	4
7、PCB 的调试：	5
二、设计环节二：秒表设计.....	5
1、要求：	5
2、电路图：	6
3、程序及实现功能：	6
4、流程图：	9
5、仿真图	12
6、计时误差：	13
7、总结：	14
三、设计环节三：交通灯控制器设计.....	14
1、要求：	14
2、电路图：	14
3、程序及实现功能：	14
4、流程图：	20
5、仿真图：	24
6、总结：	25
四、设计环节四：简易数字电压表.....	25
1、要求：	25
2、电路图：	25
3、程序及实现功能：	26
4、流程图：	28
5、仿真图：	28
6、总结：	30
五、设计环节五：简易信号发生器.....	30
1、要求：	30
2、电路图：	31
3、程序及实现功能：	31
4、流程图：	35
5、仿真图：	38
6、总结.....	40
六、设计环节六：具有闹铃功能的数字钟.....	40
1、要求：.....	40
2、电路图：	41
3、程序代码：	41

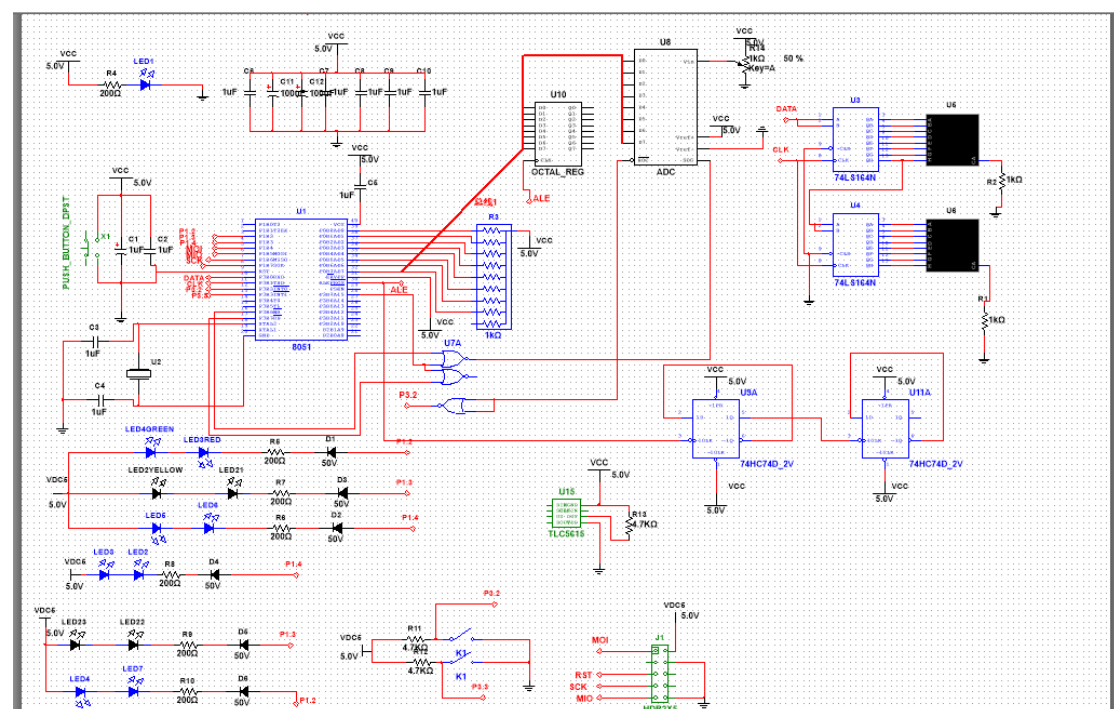
4、流程图:	47
5、仿真图	50
6、表格分析:	51
7、总结:	51
总结:	52

一、设计环节一：单片机开发板的焊接与调试

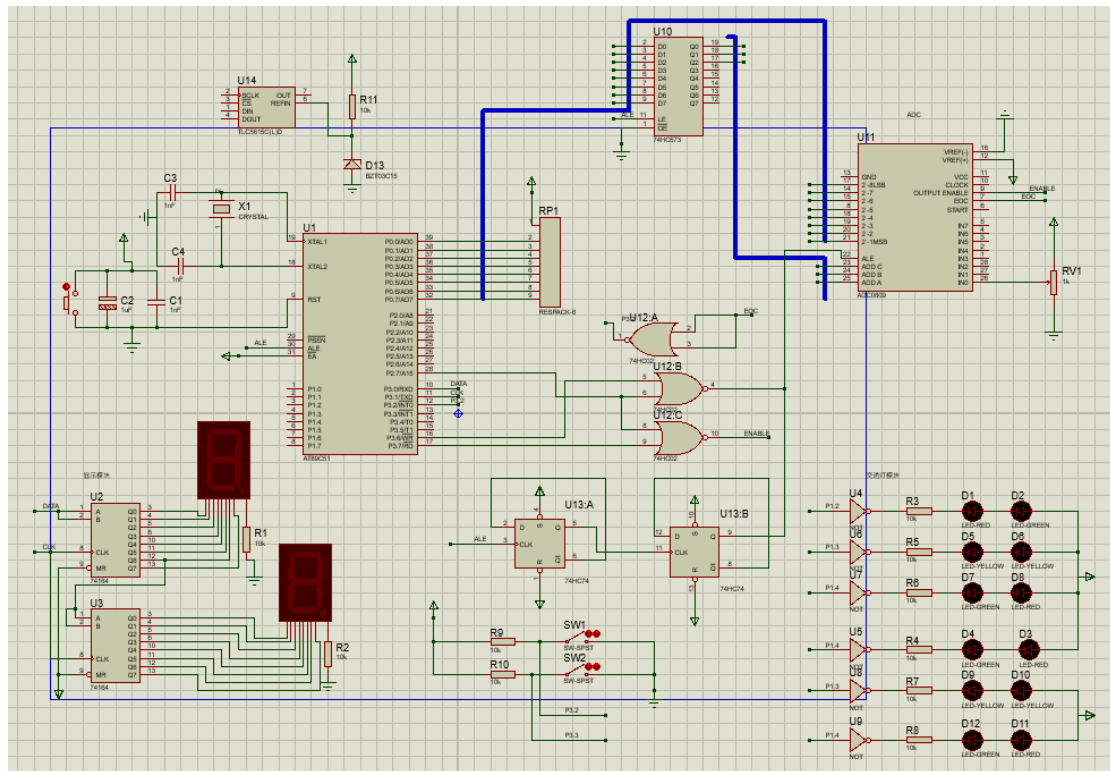
1、单片机开发板的构成框图：



2、电路图：



3、仿真图：



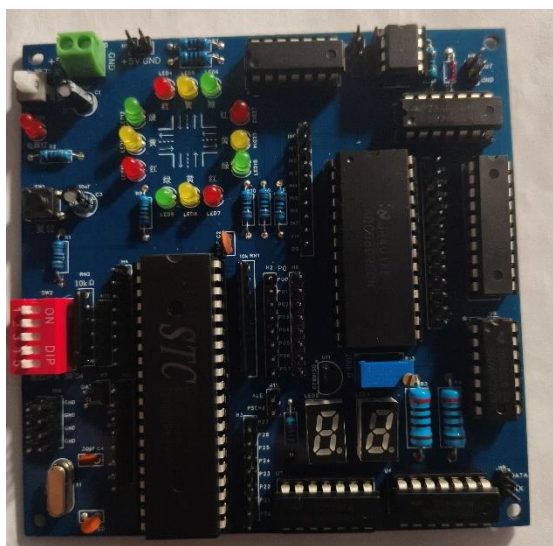
4、焊接元件清单:

器件	数量	标号	备注
KM106M025D11RR0VH2FP0	2	C1,C3	10uF
CC1-F5NP02A200KSPW	1	C2	0.1uF
CC1-F5NP02A200KSPW	2	C4,C5	30pF
BZX55C2V2	1	D1	稳压管 2.2V
P2254V-11-02P	5	Hn(n=1~16)	3 排插针 (40P / 排)
204-10SDRD/S530-A3-L	1	LED1	蓝色 LED 灯
FJ3121AH	2	LED2,LED3	数码管

器件	数量	标号	备注
204-10SDRD/S530-A3-L	4	LED4,LED7,LED13,LED16	红色 LED 灯
204-10SDRD/S530-A3-L	4	LED5,LED8,LED11,LED14	黄色 LED 灯
204-10SDRD/S530-A3-L	4	LED6,LED9,LED12,LED15	绿色 LED 灯
RN 1/4W 10K F T/B A1	2	R1,R11	10k
RN 1/4W 10K F T/B A1	1	R2	4.7k
MF1W-200Ω±1% T	2	R3,R4	200Ω
RN 1/4W 10K F T/B A1	6	R5,R6,R7,R8,R9,R10	200Ω
3296W-1-104	1	R13	100kΩ 可调电位器
RN 1/4W 10K F T/B A1	1	R14	510
A09-103JP	1	RN1	10k
A06-103JP	1	RN2	10k
TS665CJ	1	SW1	复位开关
DS-05RP	1	SW2	拨码开关
XKB5858-Z	1	SW3	电源开关
STC89C52RC-40I-PDIP40	1	U1	51 主控单片机
40 脚插座	1	U1	适配单片机
KF350-3.5-2P	1	U2	电源接口模块
74HC164N	2	U3,U4	移位寄存器
14 脚插座	5	U3,U4,U5,U8,U9	适配 14 引脚逻辑芯片
74HC04N	1	U5	六反相器

器件	数量	标号	备注
ADC0809CCN	1	U6	8 位 ADC 模数转换芯片
28 脚插座	1	U6	适配 ADC0809
SN74HC573AN	1	U7	8 位锁存器
20 脚插座	1	U7	适配 74HC573
74HC02N	1	U8	或非门
SN74HC74N	1	U9	双 D 触发器
TLC5615CP	1	U10	DAC 数模转换芯片
8 脚插座	1	U10	适配 TLC5615
DS18B20	1	U11	数字温度传感器
X49SD12MSD2SC	1	X1	12MHz 晶振

5、焊接后的开发板照片：



6、焊接过程中的注意事项：

焊接过程中要先焊接矮的元器件再焊接高的元器件。焊接时要注意焊锡的使用既不要用量过少导致接触不良，或者表面用量过多，内部虚焊。

7、PCB 的调试：

```
ORG 0000H
LJMP MAIN
ORG 0030H

MAIN: MOV P1, #04H
LCALL DELAY
MOV P1, #08H
LCALL DELAY
MOV P1, #10H
LCALL DELAY
LJMP MAIN

DELAY: MOV R5, #02H
DLO:   MOV R6, #0FFH
DL1:   MOV R7, #0FFH
DL2:   DJNZ R7, DL2
       DJNZ R6, DL1
       DJNZ R5, DLO
       RET

END
```

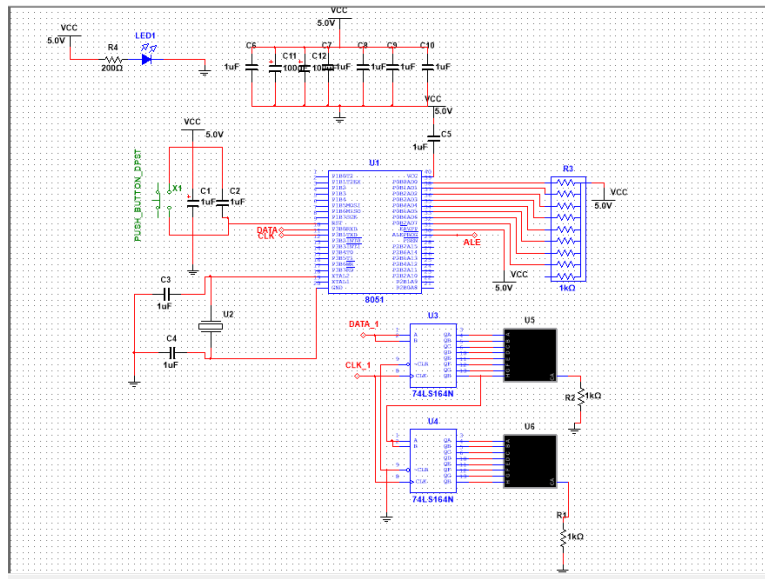
烧录代码后 PCB 现象：数码管显示 8.8., LED 灯轮流亮起。

二、设计环节二：秒表设计

1、要求：

设计一个可以从 0-60 计时，从当前计时数倒计时以及可以暂停的计时器。

2、电路图：



3、程序及实现功能：

代码：#include <reg51.h>

typedef unsigned char uchar;

typedef unsigned int uint;

uchar count = 0;

uchar i,j,mode;

uchar target_mode;

bit scmode1,scmode2,scmode3;

uint tim_flag = 0;

```
uchar code table[]={
0xFC,0x60,0xDA,0xF2,0x66,0xB6,0xBE,0xE0,
0xFE};
```

sbit mode1 = P3^2; // 当前值倒计时开关

sbit mode2 = P3^3; // 暂停开关

void delay10ms(void)

```
{
    unsigned char i,j;
    for(i=250;i>0;i--)
        for(j=248;j>0;j--);
}
```

void putch(uchar n)

```

{
    SBUF = n;
    while(TI == 0);
    TI = 0;
}

void display(void)
{
    delay10ms();
    i = table[count/10];
    j = table[count%10];
    putch(i);
    putch(j);
}

void timer0(void) interrupt 1 using 1
{
    TH0 = 0xFC;
    TL0 = 0x18;
    tim_flag++;
}

void seding(void)
{
    if(scmodel == 0 && model == 0)
    {
        target_mode = 1;
        scmodel = 1;
    }
    if(model != 0)
        scmodel = 0;

    // 暂停: mode2==0 才暂停//
    if(mode2 == 0)
        mode = 3;
    else
        mode = target_mode;
}

void count_update(void)
{
    if(tim_flag < 1000)
        return;
    tim_flag = 0;
}

```

```

switch(mode)
{
    case 0: // 正向 0~60
        count++;
        if(count > 60) count = 0;
        break;
    case 1: // 当前数值倒计时
        if(count > 0) count--;
        break;

    case 2: // 暂停, 不修改数值
        break;
}
}

void main(void)
{
    SP = 0x53;

    TMOD |= 0x01;
    TH0 = 0xFC;
    TL0 = 0x18;

    SCON = 0x00;

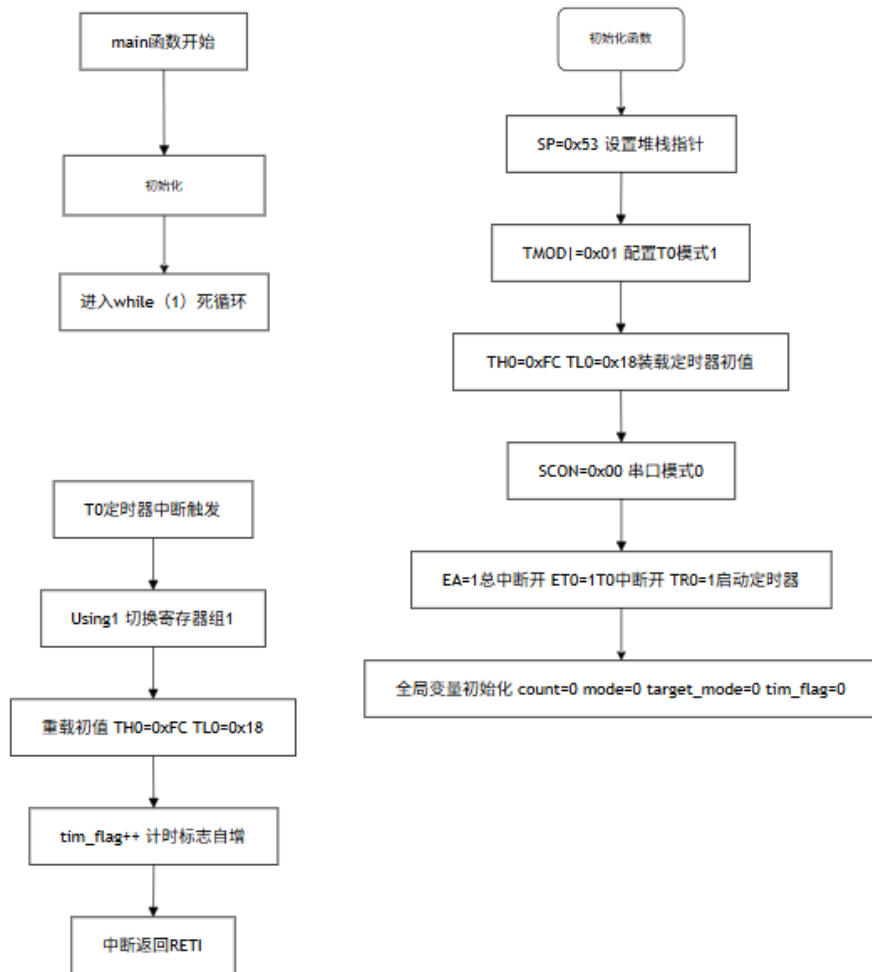
    EA = 1;
    ET0 = 1;
    TR0 = 1;

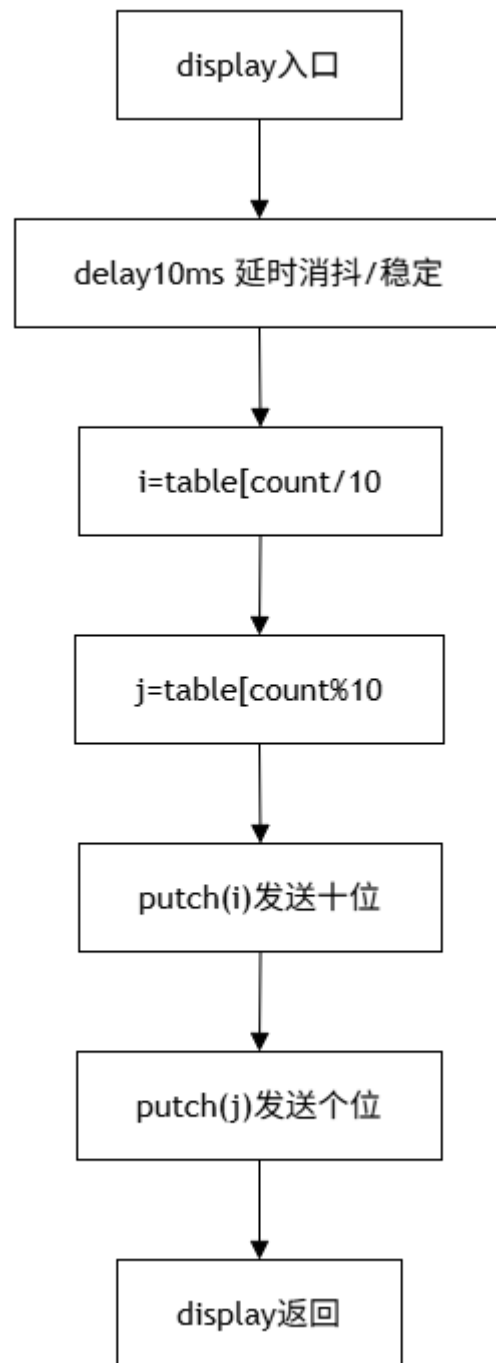
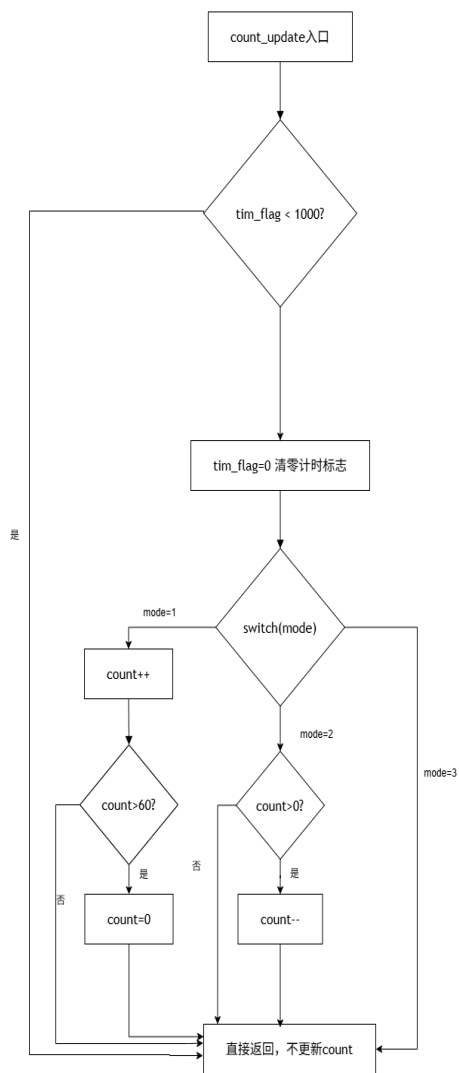
    count = 0;
    mode = 0;
    target_mode = 0;
    tim_flag = 0;

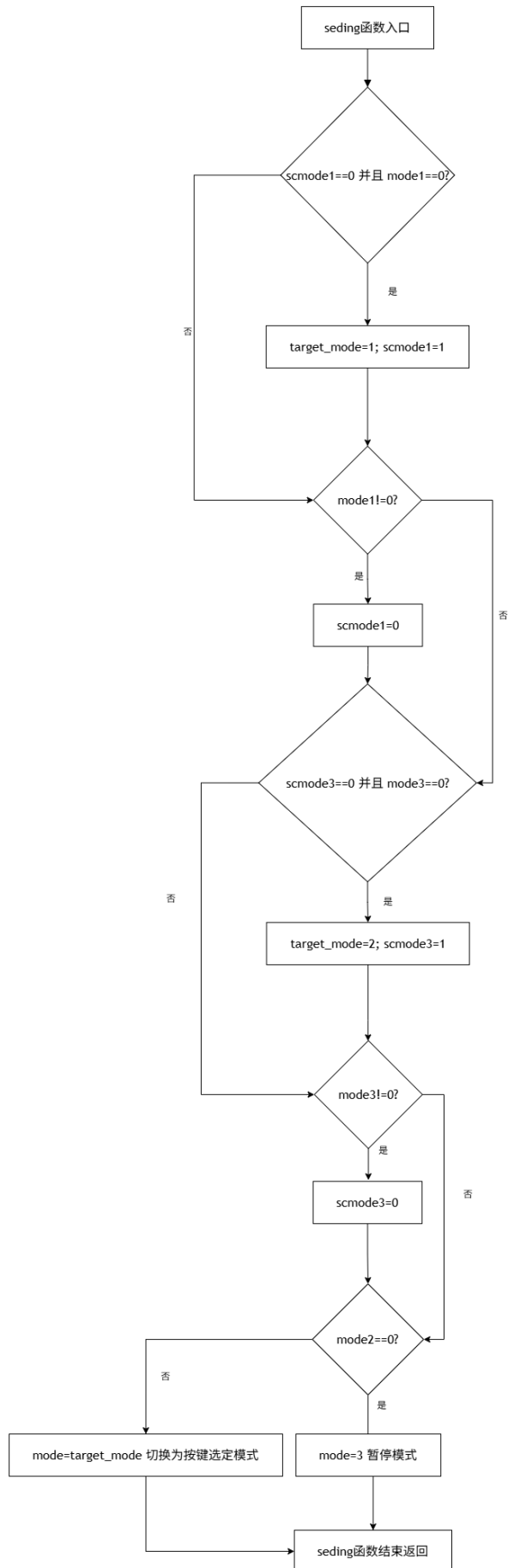
    while(1)
    {mode=0;
        seding();
        count_update();
        display();
    }
}

```

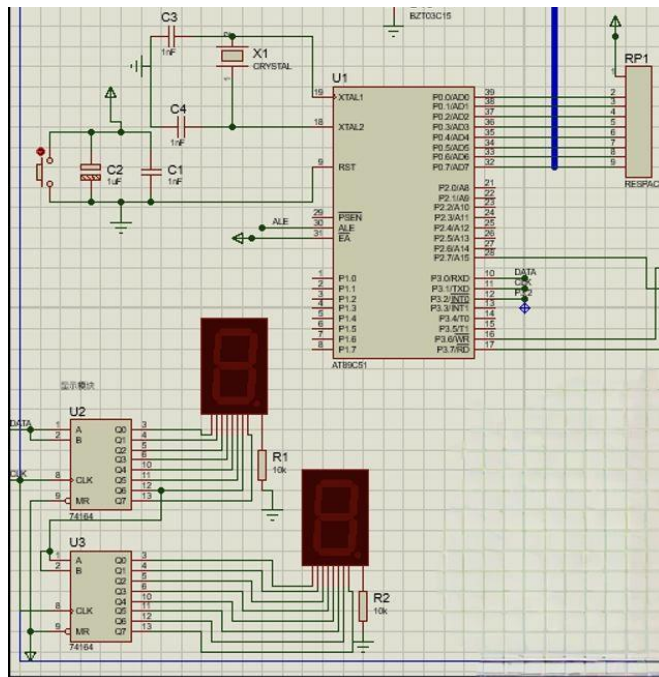
4、流程图：



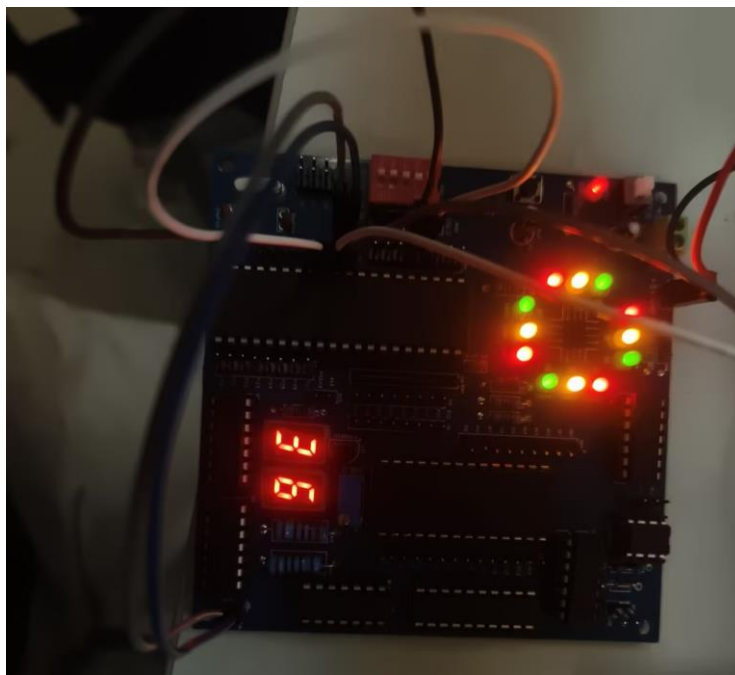


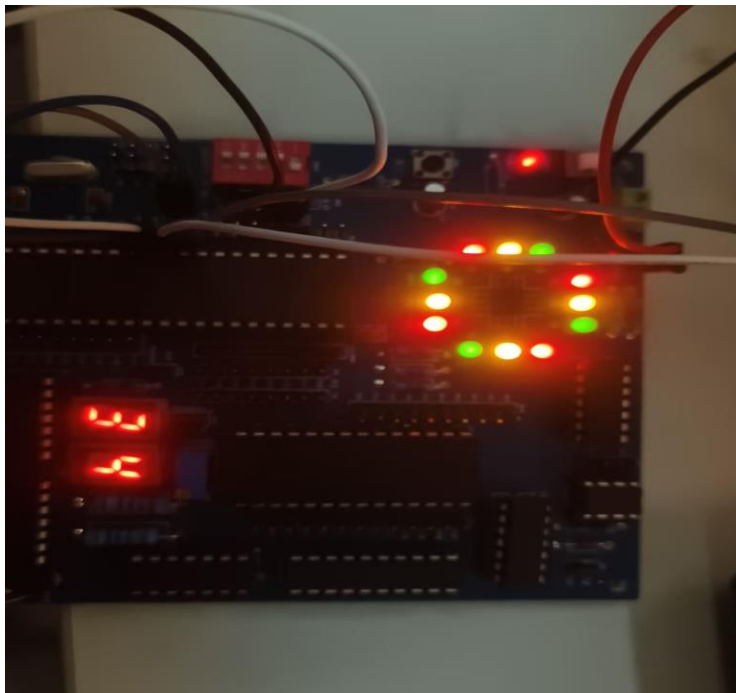
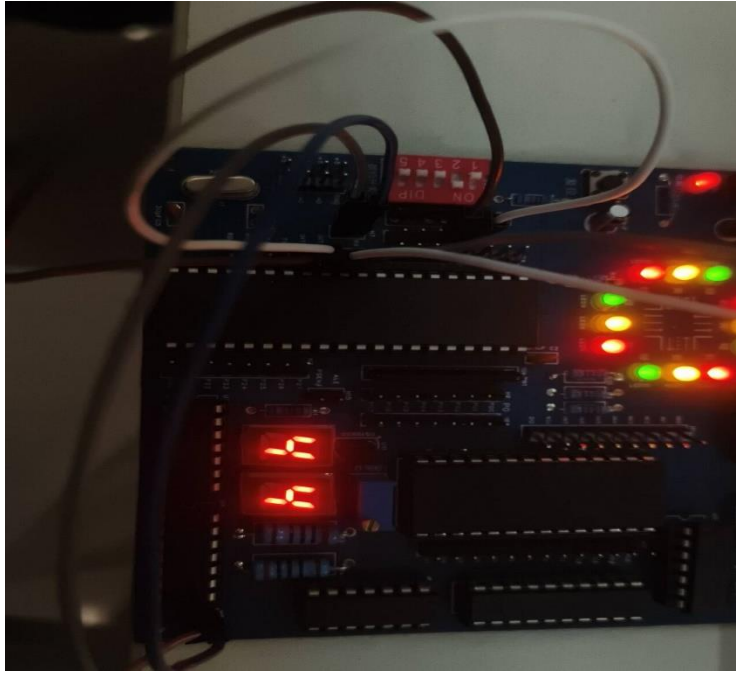


5、仿真图



现象：可以从 0-60 计时，从当前计时数倒计时以及可以暂停的计时器。





6、计时误差：

手机计时	数码管显示
0	0
05	05
27	27

7、总结：

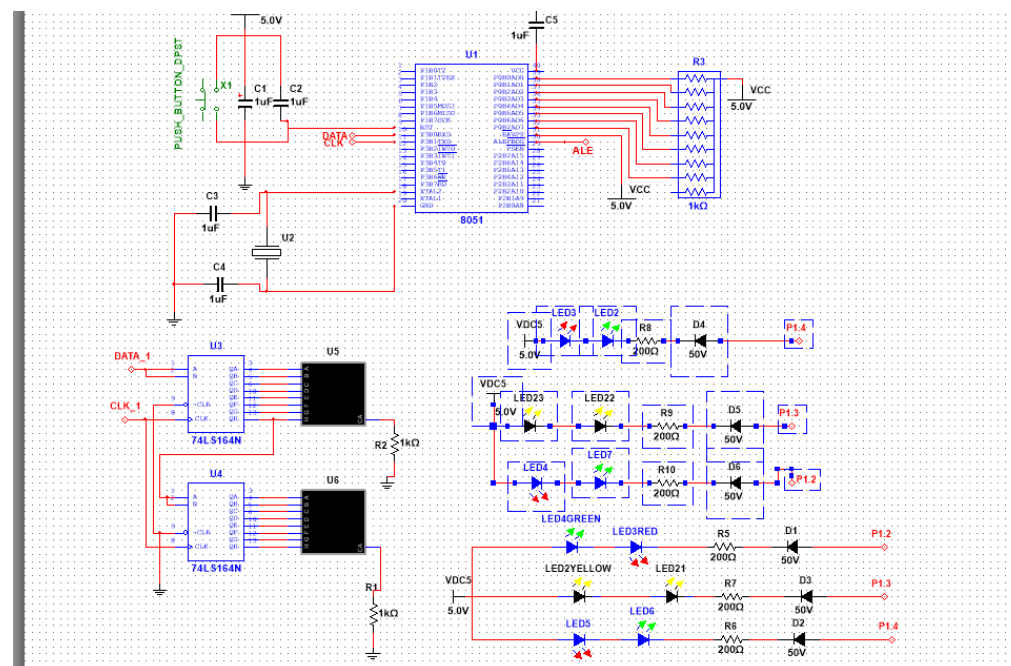
这个程序实现了从 0-60 的计数，并且可以从当前数开始倒计时以及暂停，且根据误差检验可以看出再一定计时范围内没有误差。

三、设计环节三：交通灯控制器设计

1、要求：

正常情况下南北红灯，东西绿灯，数码管 20s 倒计时；南北与东西都是黄灯，数码管 5s 倒计时；南北绿灯，东西红灯，数码管 20s 倒计时。这三种状态轮流切换。当按下按键一时，无论现在是什么状态，都切换到南北绿灯，东西红灯，数码管停在当前数字；当按键二按下时，无论现在是什么状态，都切换到，南北红灯，东西绿灯，数码管暂停在当前的数字。

2、电路图:



3、程序及实现功能:

```
#include <reg51.h>
#define uchar unsigned char
typedef unsigned int    uint;
uchar count, mode, stopflag, i, j;
uchar table[] = {0xFC,0x60,0xDA,0xF2,0x66,0xB6,0xBE,0xE0,0xFE,0xF6};
uchar sc, s;
uint tim  flag;
```

```
uchar save_s, save_count;
```

```
sbit mode1 = P1^2;
```

```
sbit mode3 = P1^3;
```

```
sbit mode2 = P1^4;
```

```
sbit nanbei = P3^2;
```

```
sbit dongxi = P3^3;
```

```
void delay10ms(void)
```

```
{  
    unsigned char i,j;  
    for(i=250; i>0; i--)  
        for(j=248; j>0; j--);  
}
```

```
void timer0(void) interrupt 1 using 1
```

```
{  
    TH0 = 252;  
    TL0 = 24;  
    tim_flag++;  
}
```

```
void putch(uchar n)
```

```
{  
    SBUF = n;  
    while(TI == 0);  
    TI = 0;  
}
```

```
void display(void)
```

```
{  
    i = table[count / 10];  
    j = table[count % 10];  
    putch(i);  
    putch(j);  
}
```

```
void count_update(void)
```

```
{  
  
    if(tim_flag < 1000)  
        return;  
    tim_flag = 0;
```

```

if(stopflag == 1)
{
    return;
}

if(s == 1)
{
    if(count > 0)
    {
        count--;
    }
    else
    {
        s = 3;
        sc = 1;
        count = 5;
    }
}

else if(s == 2)
{
    if(count > 0)
    {
        count--;
    }
    else
    {
        s = 3;
        sc = 2;
        count = 5;
    }
}

else if(s == 3)
{
    if(count > 0)
    {
        count--;
    }
    else
    {
        if(sc == 2)
        {

```

```

        s = 1;
        count = 20;
    }

    else if(sc == 1)
    {
        s = 2;
        count = 20;
    }
}
}
}

```

```

void clight(void)
{
    if(s == 1)
    {
        mode1 = 1;
        mode2 = 0;
        mode3 = 0;
    }
    else if(s == 2)
    {
        mode1 = 0;
        mode2 = 1;
        mode3 = 0;
    }
    else if(s == 3)
    {
        mode1 = 0;
        mode2 = 0;
        mode3 = 1;
    }
}
}

```

```

void jingji(void)
{
    if(nanbei == 0 || dongxi == 0)
    {
        delay10ms();
        if(nanbei == 0 || dongxi == 0)
        {
            if(stopflag == 0)
            {

```

```

        save_s = s;
        save_count = count;
        stopflag = 1;
    }

    if(nanbei == 0)
    {
        mode1 = 0;
        mode2 = 1;
        mode3 = 0;
    }
    else if(dongxi == 0)
    {
        mode1 = 1;
        mode2 = 0;
        mode3 = 0;
    }
}

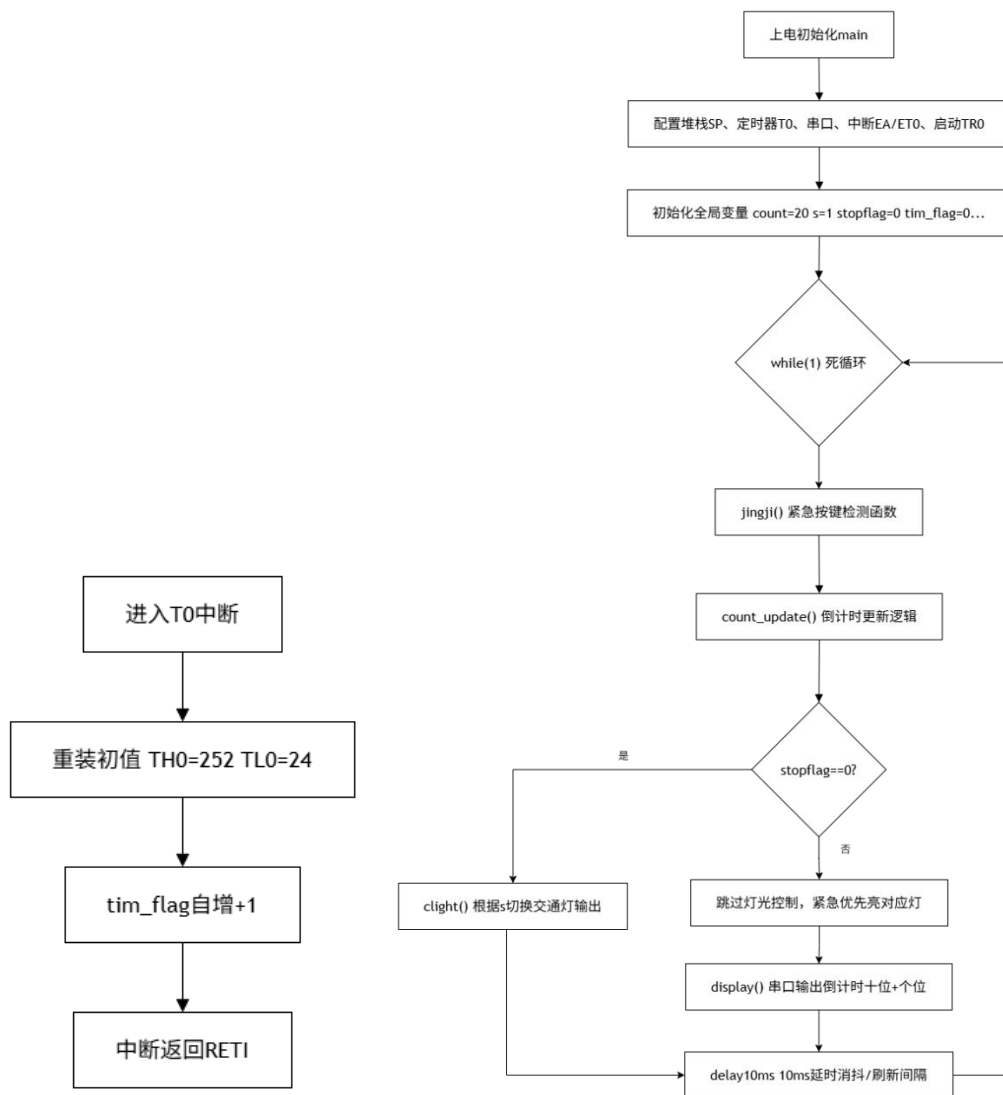
else
{
    if(stopflag == 1)
    {
        stopflag = 0;
        s = save_s;
        count = save_count;
    }
}

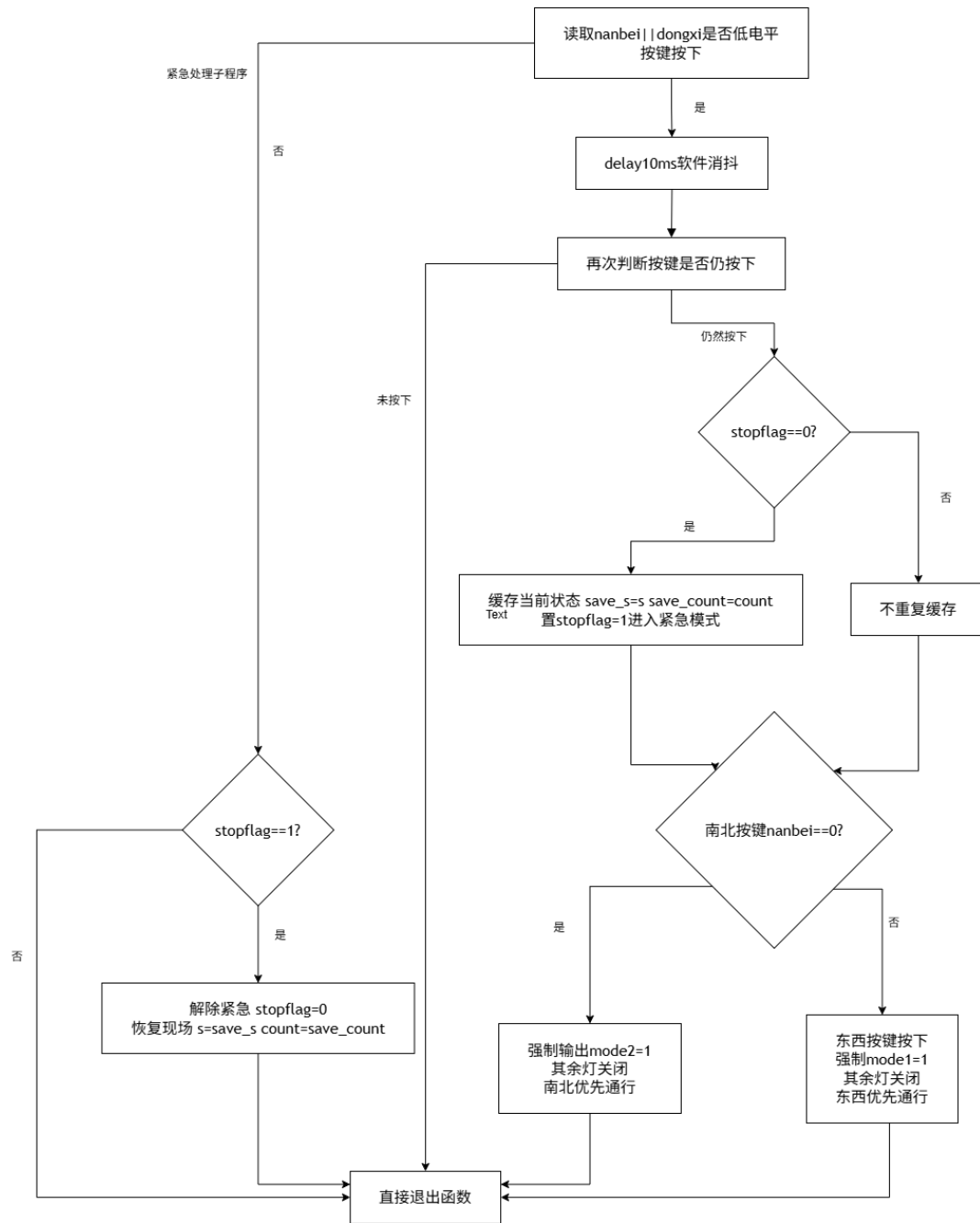
void main(void)
{
    SP = 83;
    TMOD |= 1;
    TH0 = 252;
    TL0 = 24;
    SCON = 0;
    EA = 1;
    ET0 = 1;
    TR0 = 1;
    count = 20;
    s = 1;
    stopflag = 0;
}

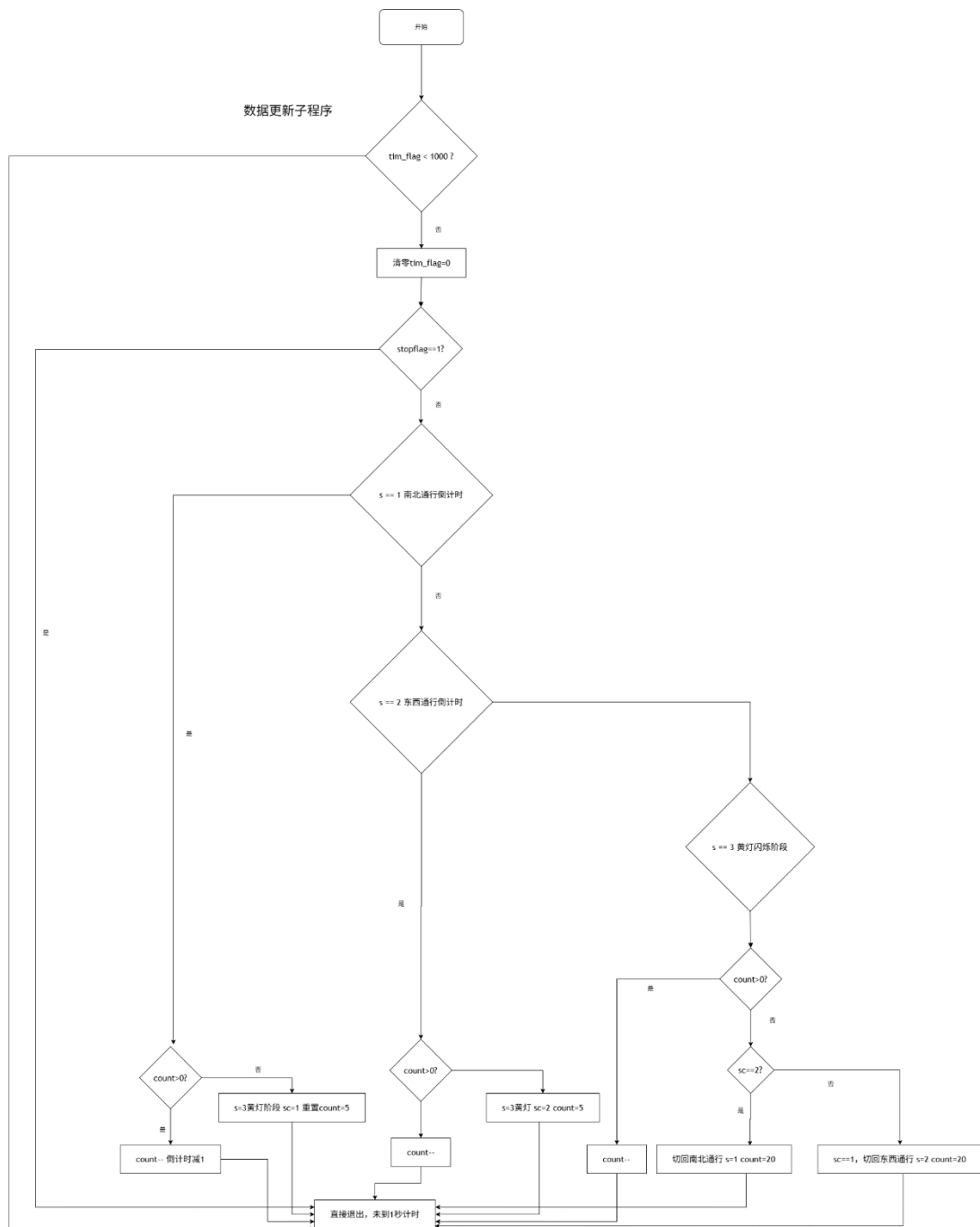
```

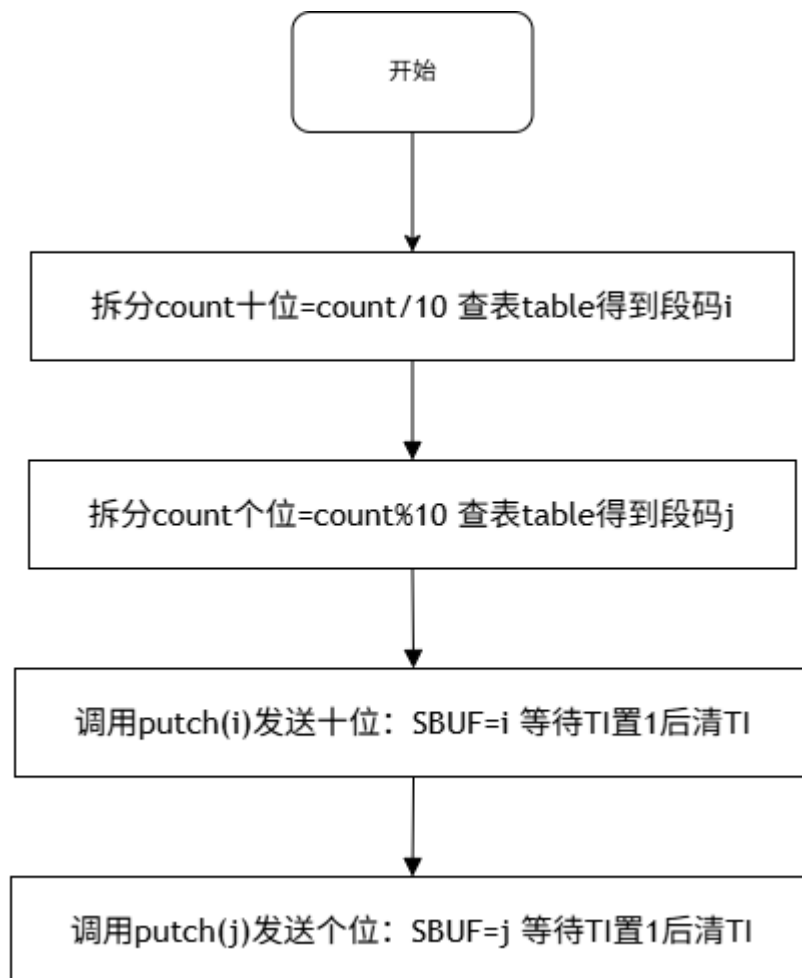
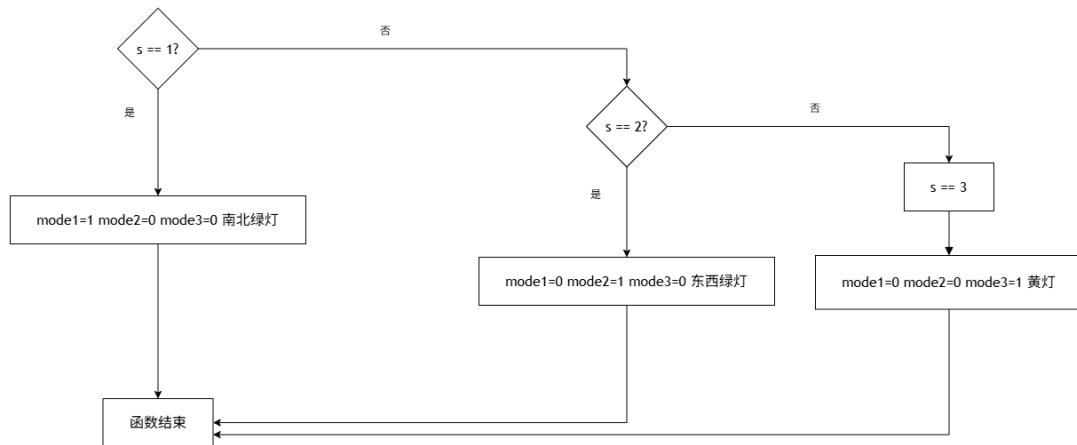
```
tim_flag = 0;
sc = 0;
save_s = 1;
save_count = 20;
while(1)
{
    jingji();
    count_update();
    if(stopflag == 0)
    {
        clight();
    }
    display();
    delay10ms();
}
}
```

4、流程图：

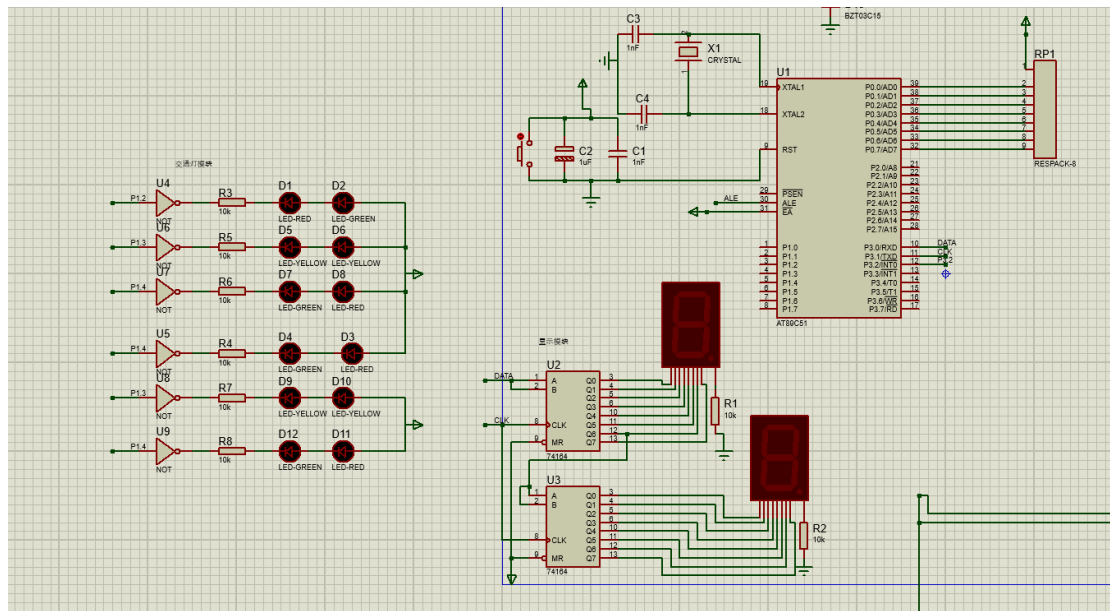




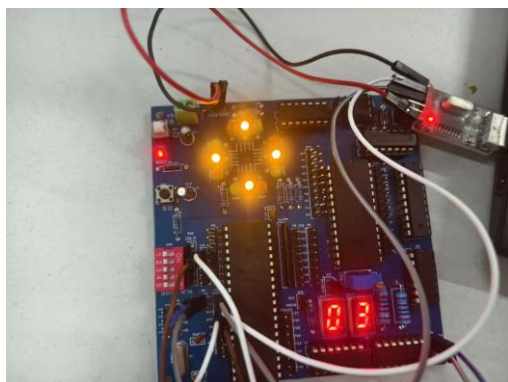
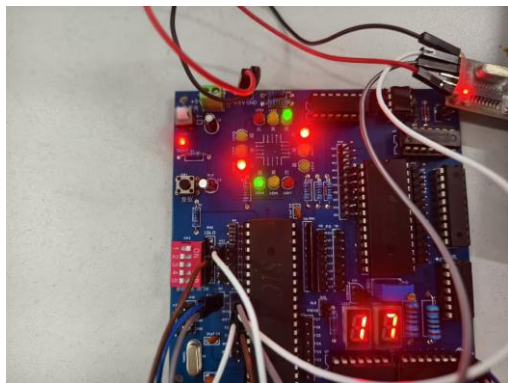


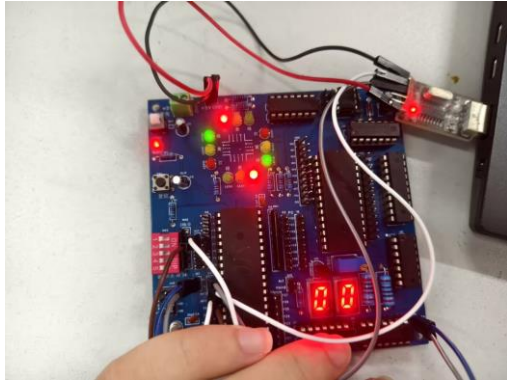


5、仿真图：



现象：正常情况下南北红灯，东西绿灯，数码管 20s 倒计时；南北与东西都是黄灯，数码管 5s 倒计时；南北绿灯，东西红灯，数码管 20s 倒计时。这三种状态轮流切换。当按下按键二时，切换到南北绿灯，东西红灯，数码管停在 00；当按键一按下时，切换到，南北红灯，东西绿灯，数码管暂停在 17。





6、总结：

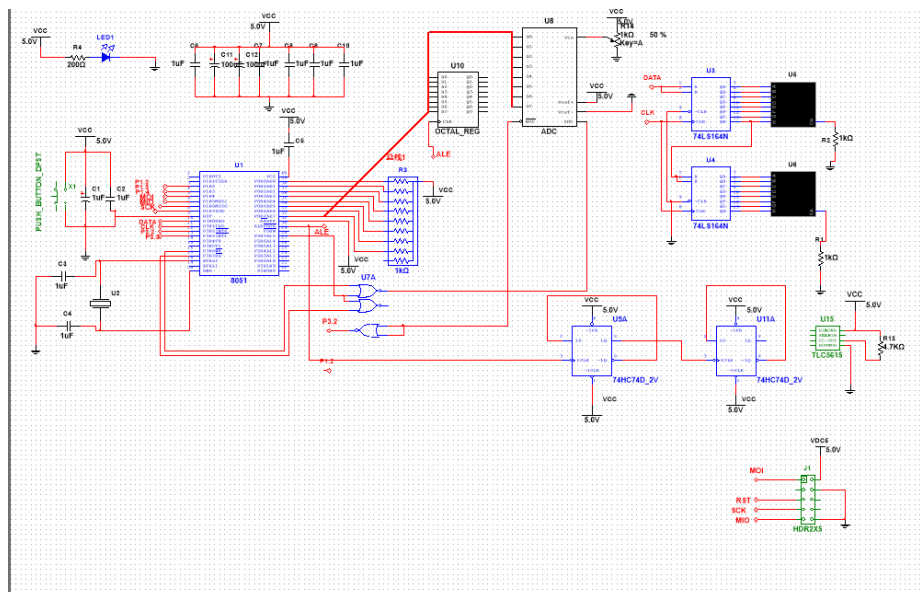
本次设计实现了正常情况下南北红灯, 东西绿灯, 数码管 20s 倒计时; 南北与东西都是黄灯, 数码管 5s 倒计时; 南北绿灯, 东西红灯, 数码管 20s 倒计时。这三种状态轮流切换。当按下按键一时, 无论现在是什么状态, 都切换到南北绿灯, 东西红灯, 数码管停在当前数字; 当按键二按下时, 无论现在是什么状态, 都切换到, 南北红灯, 东西绿灯, 数码管暂停在当前的数字。

四、设计环节四：简易数字电压表

1、要求：

设计一个数字电压表, 数码管显示电压值, 精度为一位小数。

2、电路图：



3、程序及实现功能：

```
#include <reg51.h>
#include <intrins.h>
#include <absacc.h>
#include <stdio.h>
#define uchar unsigned char
#define uint unsigned int
#define IN0 0X7000
uchar addata;
uchar p,q;
uchar code TAB[]={0xFC,0x60,0xDA,0xF2,0x66,0xB6,0xBE,0xE0,0xFE,0xF6};

void mydelay(uint n)
{
    uchar i;
    while(n--)
        for(i=150;i>0;i--);
}

void putch(uchar n)
{
    SBUF=n;
    while(TI==0);
    TI=0;
}

void jisuan(uchar ad)
{
    uint temp = (uint)ad * 5;
    p = temp / 255;
    q = (temp % 255) * 10 / 255;
}

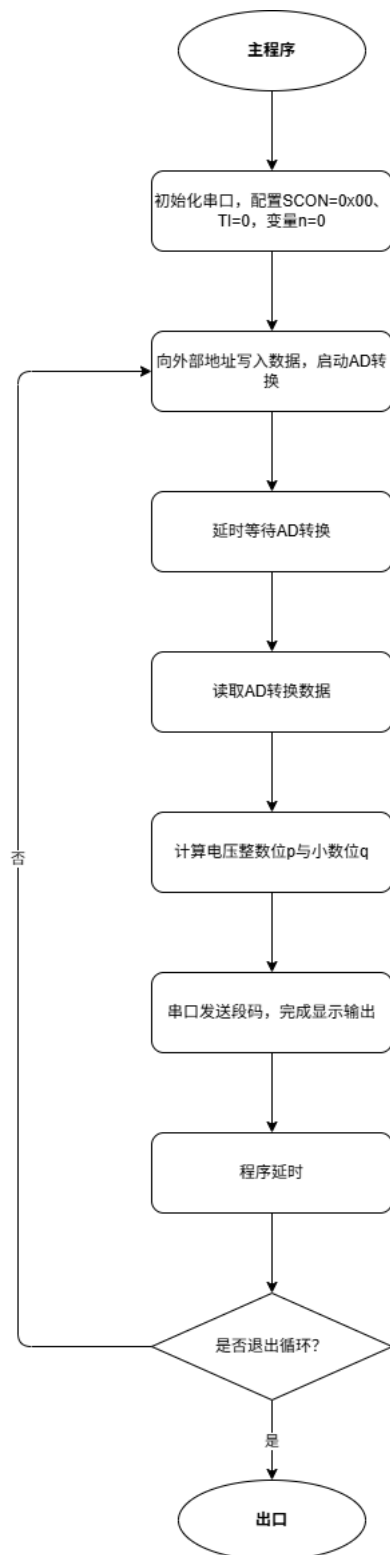
void mydisplay()
{
    putch(TAB[p] | 0x80);
    putch(TAB[q]);
    mydelay(200);
}

void main()
{
    uchar n=0;
```

```
SCON=0x00;
TI=0;

while(1)
{
    XBYTE[IN0+n]=0;
    mydelay(300);
    adddata = XBYTE[IN0+n];
    jisuan(adddata);
    mydisplay();
    mydelay(500);
}
}
```

4、流程图：



5、仿真图：



6、总结：

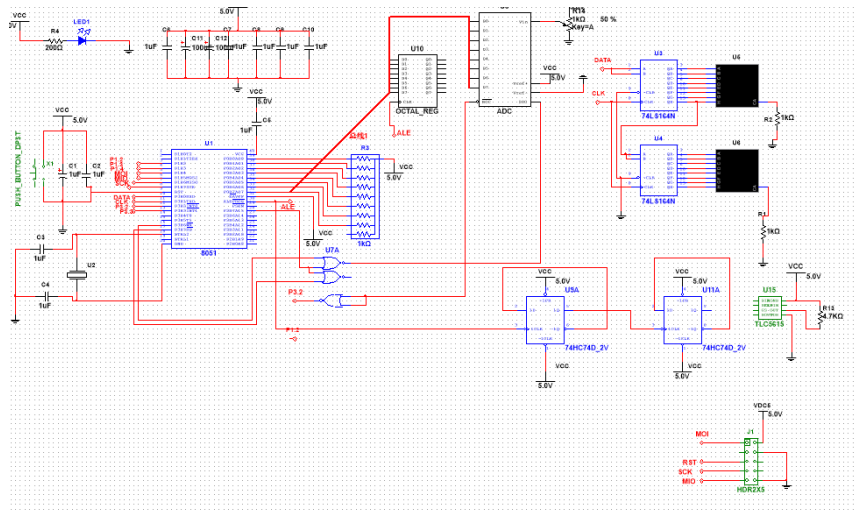
本次设计实现了数字电压表的功能，增强了对单片机程序设计的理解，以及如何调试程序。

五、设计环节五：简易信号发生器

1、要求：

设计一个程序实现：用按键控制发生方波、三角波、锯齿波、梯形波、正弦波五个波形，并且可以通过调节电位器实现在振幅不变的前提下改变频率。

2、电路图：



3、程序及实现功能:

```
#include<reg51.h>
#include <math.h>
#include "absacc.h"
#define uchar unsigned char
#define uint    unsigned int

#define AD_BASE 0X7000

sbit CS=P2^0;
sbit SCLK=P2^1;
sbit DIN=P2^2;

sbit mode_fangbo = P1^0;
sbit mode_juchibo = P1^1;
sbit mode_tixingbo = P1^2;
sbit mode_sanjiaobo = P1^3;
sbit mode_zhengxianbo = P1^4;

uint i;
uint cycle_points;
uint fuzhi;
uchar ad_val0;
```

```
void Delay(unsigned int xms)
{
    unsigned int i,j;
```

```

        for(i=xms;i>0;i--)
            for(j=124;j>0;j--);
    }

void TLC5615_Conv(unsigned int dat)
{
    unsigned char i;
    if(dat > 1000) dat = 1000;
    dat <=< 6;
    SCLK = 0;
    CS = 1;
    CS = 0;
    for(i=0;i<12;i++)
    {
        DIN = (bit)(dat & 0x8000);
        SCLK = 1;
        dat <=< 1;
        SCLK = 0;
    }
    CS = 1;
}

uchar ADC_Read(uchar ch)
{
    XBYTE[AD_BASE + ch] = 0;
    Delay(1);
    return XBYTE[AD_BASE + ch];
}

void main()
{
    P1 = 0xFF;
    fuzhi = 1000;

    while(1)
    {

        MAIN_LOOP:

        ad_val0 = ADC_Read(0);
        cycle_points = 50 + (uint)ad_val0 * 1950 / 255;

        if (mode_fangbo == 0)
        {

```

```

TLC5615_Conv(fuzhi);
for(i=0; i<cycle_points/2; i++)
{
    Delay(1);
    if(mode_fangbo != 0) goto MAIN_LOOP;
}

TLC5615_Conv(0);
for(i=0; i<cycle_points/2; i++)
{
    Delay(1);
    if(mode_fangbo != 0) goto MAIN_LOOP;
}
}
else if (mode_juchibo == 0)
{
    for (i=0;i<cycle_points;i++)
    {

        TLC5615_Conv( (uint)((unsigned long)fuzhi * i / (cycle_points-1)) );
        Delay(1);
        if(mode_juchibo != 0) goto MAIN_LOOP;
    }
}
else if (mode_tixingbo == 0)
{
    uint seg1 = cycle_points / 4;
    uint seg2 = cycle_points / 2;
    uint seg3 = cycle_points / 4;

    for(i=0;i<seg1;i++)
    {
        TLC5615_Conv( (uint)((unsigned long)fuzhi * i / (seg1-1)) );
        Delay(1);
        if(mode_tixingbo != 0) goto MAIN_LOOP;
    }

    for(i=0;i<seg2;i++)
    {
        TLC5615_Conv(fuzhi);
        Delay(1);

```

```

        if(mode_tixingbo != 0) goto MAIN_LOOP;
    }

    for(i=0;i<seg3;i++)
    {
        TLC5615_Conv( (uint)((unsigned long)fuzhi * (seg3-1-i) / (seg3-1)) );
        Delay(1);
        if(mode_tixingbo != 0) goto MAIN_LOOP;
    }
}
else if (mode_sanjiaobo == 0)
{
    uint half_cycle = cycle_points / 2;

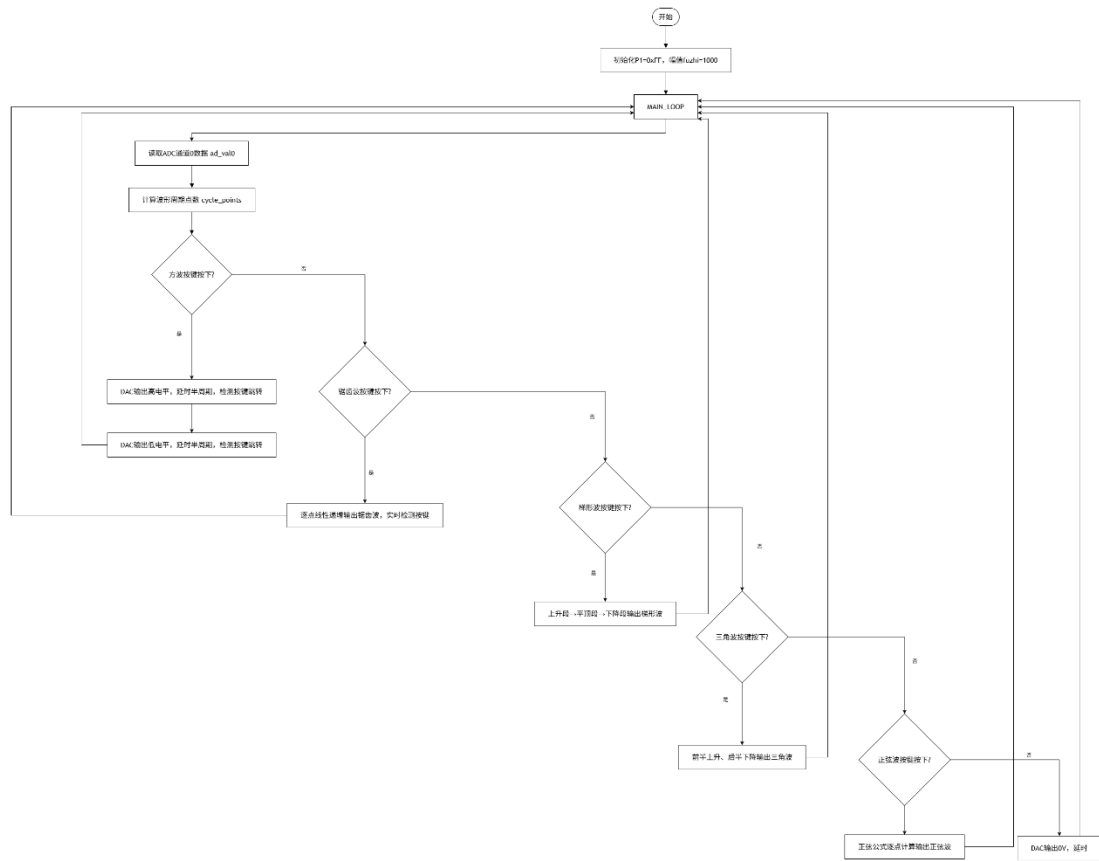
    for(i=0;i<half_cycle;i++)
    {
        TLC5615_Conv( (uint)((unsigned long)fuzhi * i / (half_cycle-1)) );
        Delay(1);
        if(mode_sanjiaobo != 0) goto MAIN_LOOP;
    }

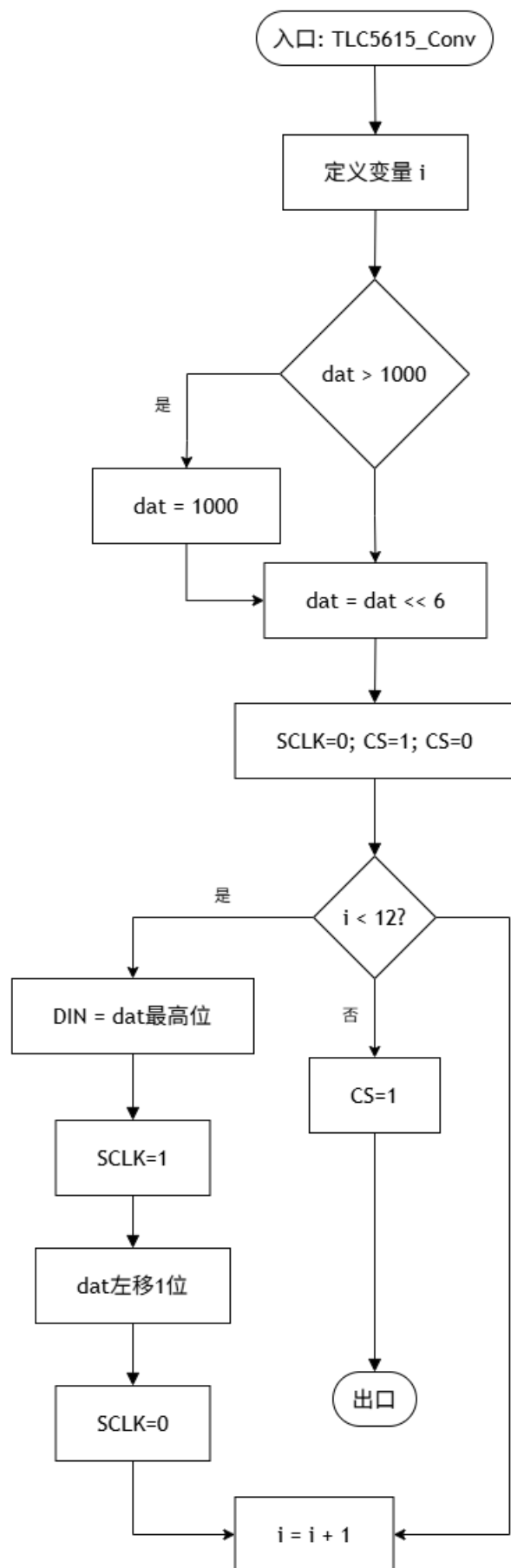
    for(i=0;i<half_cycle;i++)
    {
        TLC5615_Conv( (uint)((unsigned long)fuzhi * (half_cycle-1-i) / (half_cycle-
1)) );

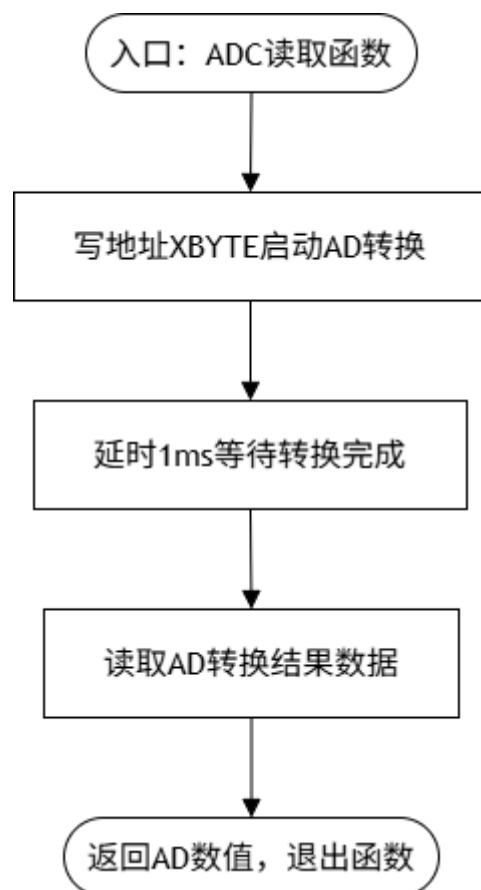
        Delay(1);
        if(mode_sanjiaobo != 0) goto MAIN_LOOP;
    }
}
else if (mode_zhengxianbo == 0)
{
    for(i=0;i<cycle_points;i++)
    {
        float sin_data = sin(2 * 3.1415926 * i / cycle_points);
        uint wave_out = (uint)(fuzhi * (sin_data + 1) / 2);
        TLC5615_Conv(wave_out);
        Delay(1);
        if(mode_zhengxianbo != 0) goto MAIN_LOOP;
    }
}
else
{
    TLC5615_Conv(0);
    Delay(1);
}

```

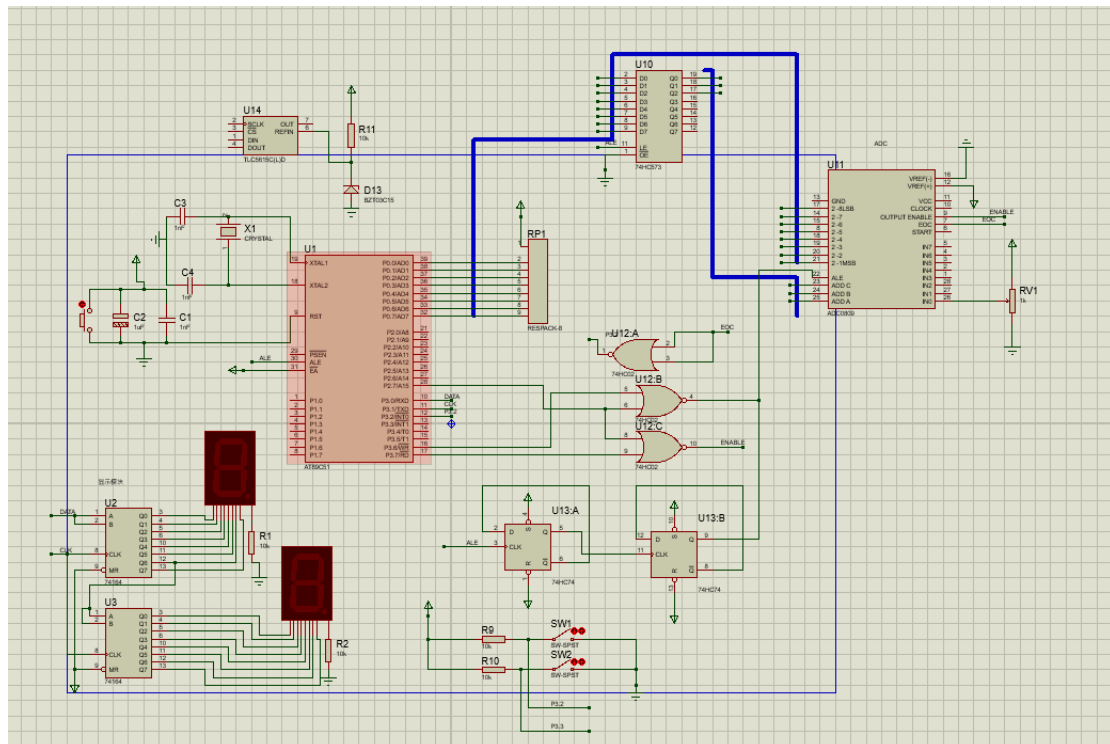
4、流程图:



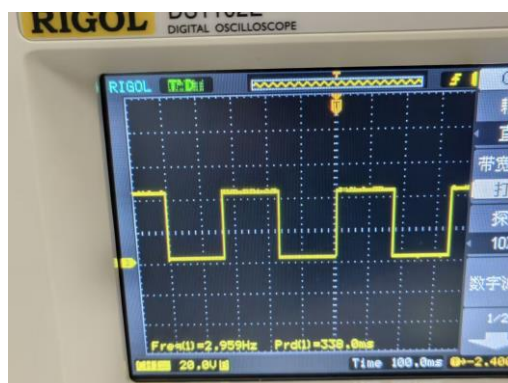
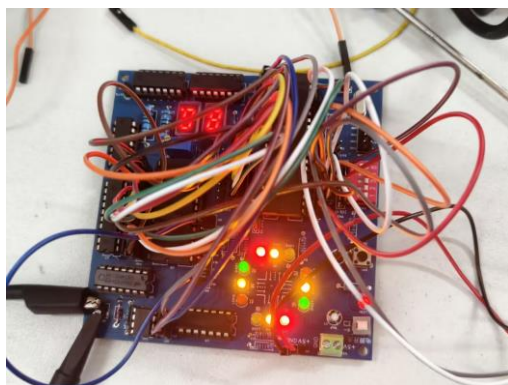


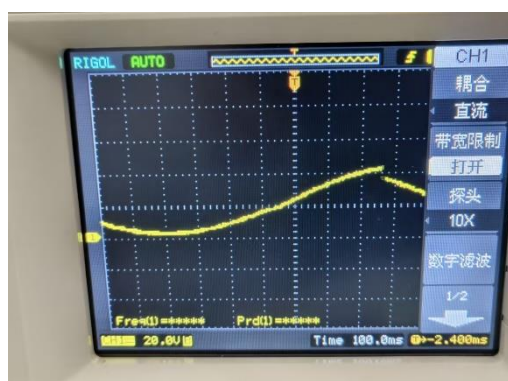
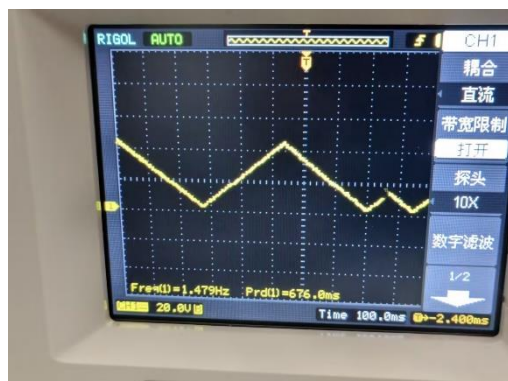
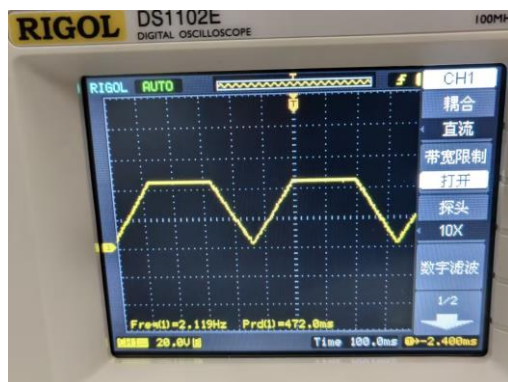


5、仿真图:

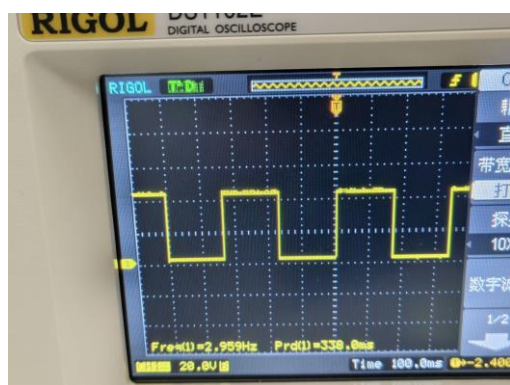
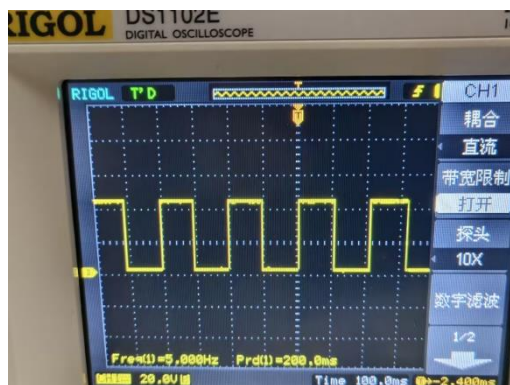


现象：实现了用按键控制发生方波、三角波、锯齿波、梯形波、正弦波五个波形，并且可以通过调节电位器实现在振幅不变的前提下改变频率。





频率调整前后：



6、总结

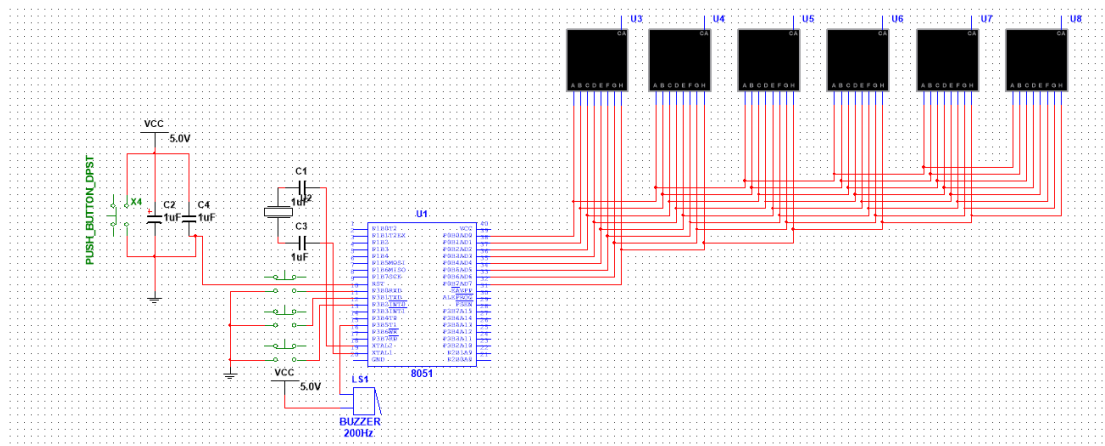
本次设计增强了我对单片机程序设计的理解，增强了我调试代码的能力，熟悉了 TLC5615 的作用，以及利用现有的设备完成设计的能力。

六、设计环节六：具有闹铃功能的数字钟

1、要求：

六位数码管，显示时，分，秒，使用一个按键调节每一位的时间，设定闹钟，到达闹钟设定时间后蜂鸣器响起。

2、电路图：



3、程序代码：

```
#include <reg51.h>
#define uchar unsigned char
#define uint unsigned int

#define FOSC 12000000UL
#define T0_PERIOD 2000UL
#define T0_INIT (65536UL - (FOSC/12UL * T0_PERIOD/1000000UL))

sbit KEY_MODE = P3^0;
sbit KEY_UP = P3^1;
sbit KEY_DOWN = P3^2;
sbit BUZZER = P3^5;

uchar code segTab[] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66,
    0x6D, 0x7D, 0x07, 0x7F, 0x6F
};

volatile uchar dispBuf[6];
#define BIT_ACTIVE 0

uchar hour = 12, minute = 0, second = 0;
uchar alarmHour = 0, alarmMinute = 0;
bit alarmEnable = 0;
bit alarmFlag = 0;

uchar mode = 0;

volatile uint tick = 0;
```

```

volatile uint secTick = 0;
bit buzzOut = 0;

void UpdateDisplay()
{
    uchar i;
    for (i = 0; i < 6; i++) dispBuf[i] = 10;

    switch(mode)
    {
        case 0:
            dispBuf[0] = second % 10;
            dispBuf[1] = second / 10;
            dispBuf[2] = minute % 10;
            dispBuf[3] = minute / 10;
            dispBuf[4] = hour % 10;
            dispBuf[5] = hour / 10;
            break;
        case 1:
            dispBuf[4] = hour % 10;
            dispBuf[5] = hour / 10;
            break;
        case 2:
            dispBuf[2] = minute % 10;
            dispBuf[3] = minute / 10;
            break;
        case 3:
            dispBuf[4] = alarmHour % 10;
            dispBuf[5] = alarmHour / 10;
            break;
        case 4:
            dispBuf[2] = alarmMinute % 10;
            dispBuf[3] = alarmMinute / 10;
            break;
    }
}

void Timer0_ISR() interrupt 1
{
    static uchar dispIndex = 0;
    TH0 = T0_INIT / 256;
    TL0 = T0_INIT % 256;

    #if BIT_ACTIVE == 0

```

```

        P2 = 0xFF;
    #else
        P2 = 0x00;
    #endif

    if (dispBuf[dispIndex] <= 9)
        P0 = segTab[dispBuf[dispIndex]];
    else
        P0 = 0x00;

    #if BIT_ACTIVE == 0
        P2 = ~(1 << dispIndex);
    #else
        P2 = 1 << dispIndex;
    #endif

    tick++;
    secTick++;

    dispIndex++;
    if (dispIndex >= 6) dispIndex = 0;
}

void TimeCountProcess(void)
{
    if(secTick >= 500)
    {
        secTick = 0;
        second++;
        if (second >= 60) {
            second = 0;
            minute++;
            if (minute >= 60) {
                minute = 0;
                hour++;
                if (hour >= 24) hour = 0;
            }
        }
        if (alarmEnable && hour == alarmHour && minute == alarmMinute && second == 0)
            alarmFlag = 1;
        UpdateDisplay();
    }

    if(tick >= 125)

```

```

    {
        tick = 0;
        if(alarmFlag)
        {
            buzzOut = ~buzzOut;
            BUZZER = buzzOut;
        }
        else
        {
            BUZZER = 1;
            buzzOut = 0;
        }
    }
}

void KeyProcess()
{
    static uchar mode_cnt = 0, up_cnt = 0, down_cnt = 0;
    static bit mode_press = 0, up_press = 0, down_press = 0;

    if(KEY_MODE == 0)
    {
        if(++mode_cnt >= 20)
        {
            mode_cnt = 20;
            if(!mode_press)
            {
                mode_press = 1;
                if(alarmFlag)
                    alarmFlag = 0;
                else
                {
                    mode = (mode + 1) % 5;
                    UpdateDisplay();
                    if(mode == 0) alarmEnable = 1;
                }
            }
        }
    }
    else
    {
        mode_cnt = 0;
        mode_press = 0;
    }
}

```

```

if(KEY_UP == 0)
{
    if(++up_cnt >= 20)
    {
        up_cnt = 20;
        if(!up_press)
        {
            up_press = 1;
            if(alarmFlag)
                alarmFlag = 0;
            else
            {
                switch(mode)
                {
                    case 1: hour++; if(hour>=24) hour=0; break;
                    case 2: minute++; if(minute>=60) minute=0; break;
                    case 3: alarmHour++; if(alarmHour>=24) alarmHour=0; break;
                    case 4: alarmMinute++; if(alarmMinute>=60) alarmMinute=0; break;
                }
                UpdateDisplay();
            }
        }
    }
}
else
{
    up_cnt = 0;
    up_press = 0;
}

if(KEY_DOWN == 0)
{
    if(++down_cnt >= 20)
    {
        down_cnt = 20;
        if(!down_press)
        {
            down_press = 1;
            if(alarmFlag)
                alarmFlag = 0;
            else
            {
                switch(mode)

```



```

        {
            case 1: if(hour>0) hour--; else hour=23; break;
            case 2: if(minute>0) minute--; else minute=59; break;
            case 3: if(alarmHour>0) alarmHour--; else alarmHour=23; break;
            case 4: if(alarmMinute>0) alarmMinute--; else alarmMinute=59;
break;
        }
        UpdateDisplay();
    }
}
}
else
{
    down_cnt = 0;
    down_press = 0;
}
}

```

```

void main()
{
    P0 = 0x00;
    #if BIT_ACTIVE == 0
        P2 = 0xFF;
    #else
        P2 = 0x00;
    #endif
    BUZZER = 1;

    UpdateDisplay();

    TMOD &= 0xF0;
    TMOD |= 0x01;
    TH0 = T0_INIT / 256;
    TL0 = T0_INIT % 256;

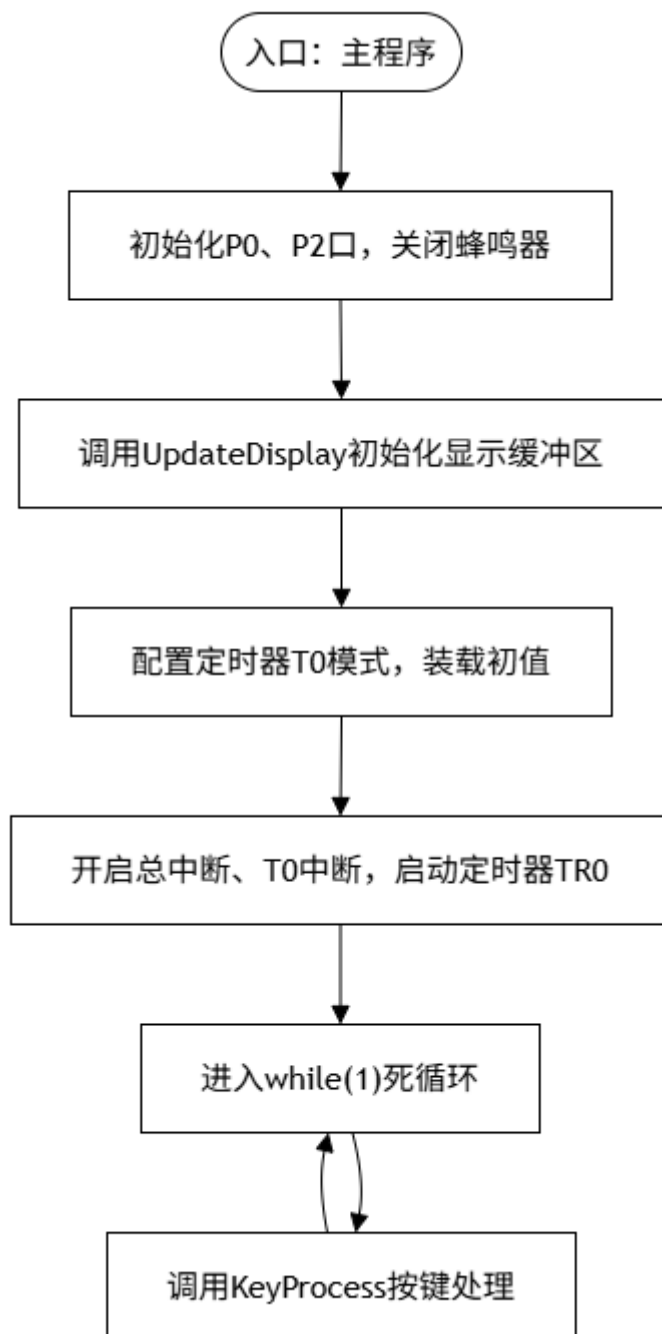
    EA = 1;
    ET0 = 1;
    TR0 = 1;

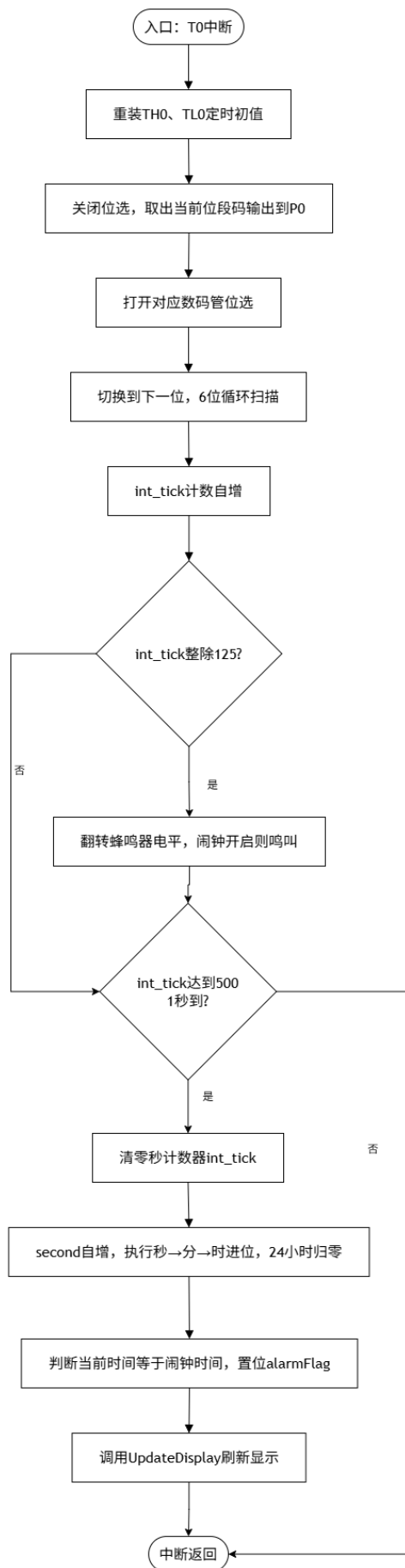
    while (1)
    {
        KeyProcess();
        TimeCountProcess();
    }
}

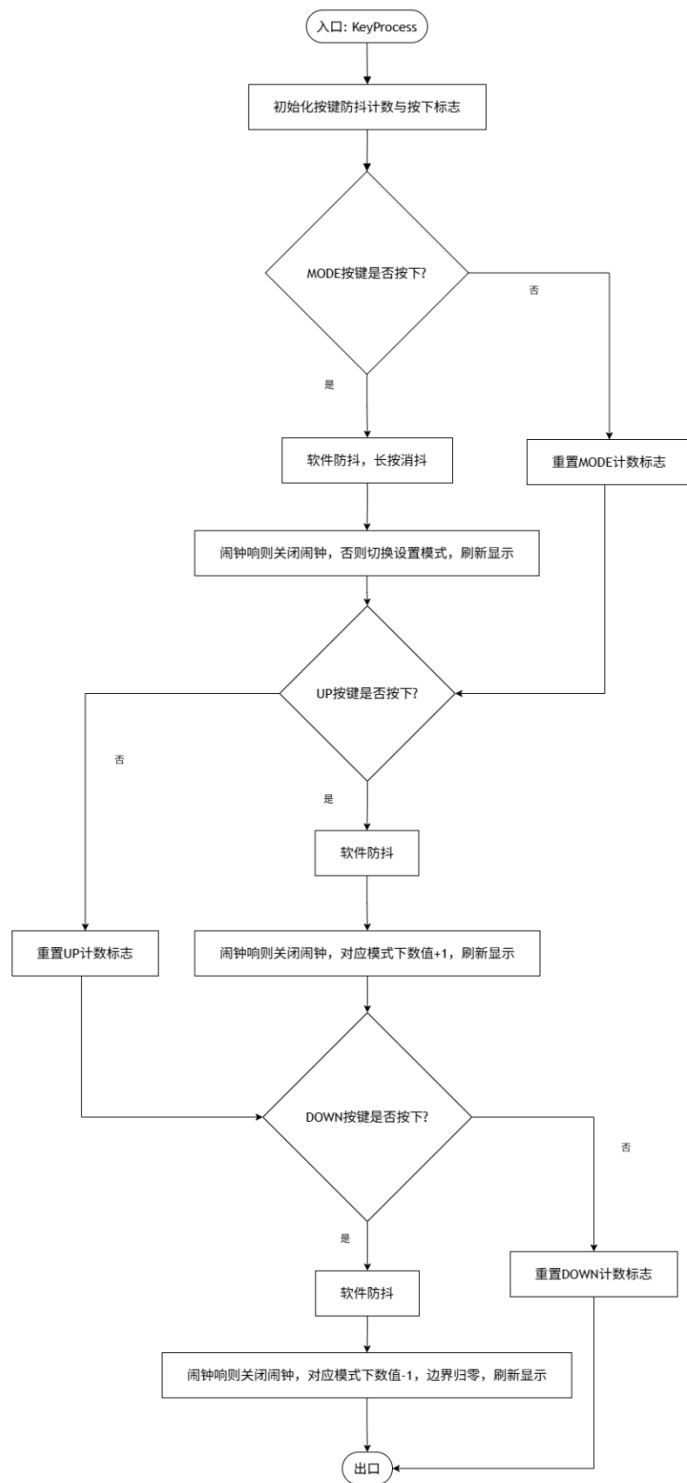
```

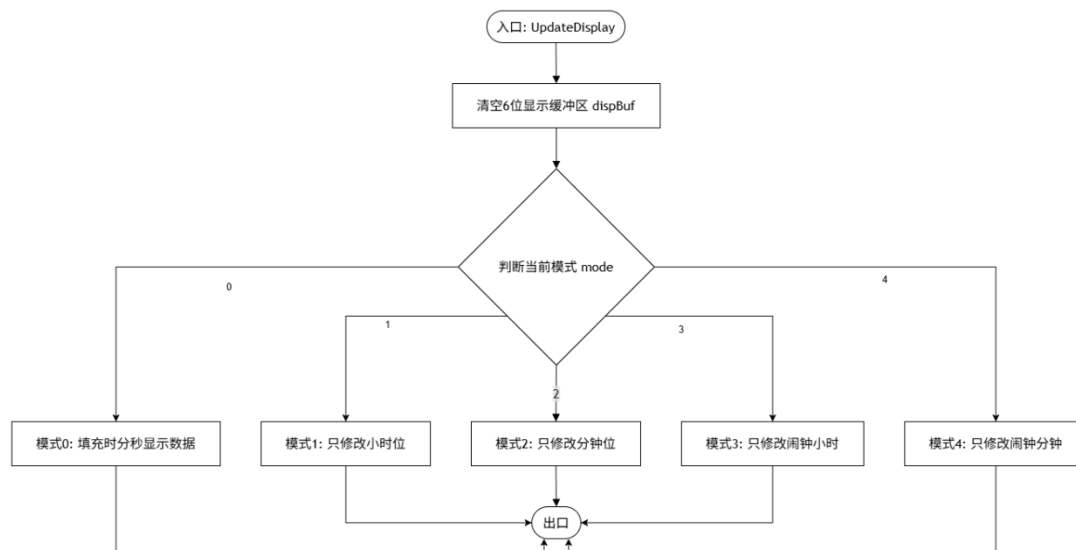
```
}  
}
```

4、流程图：

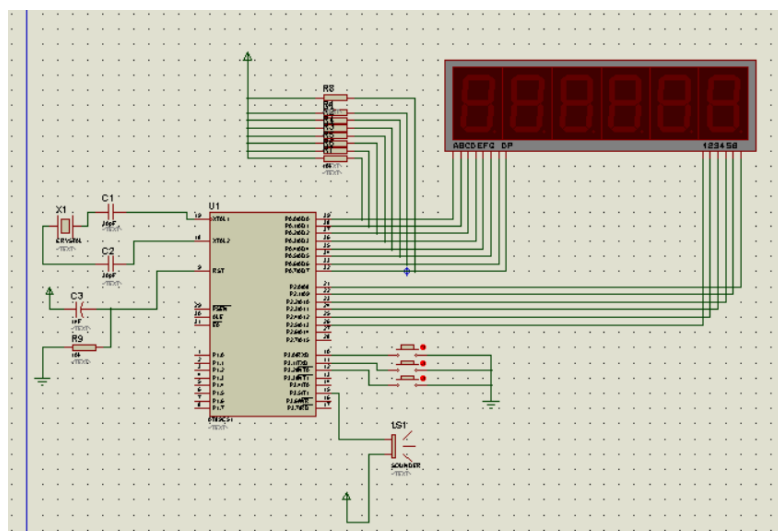




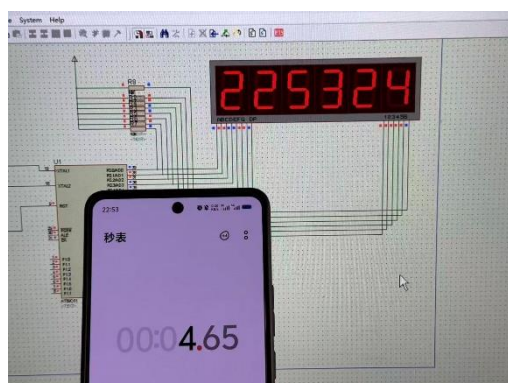


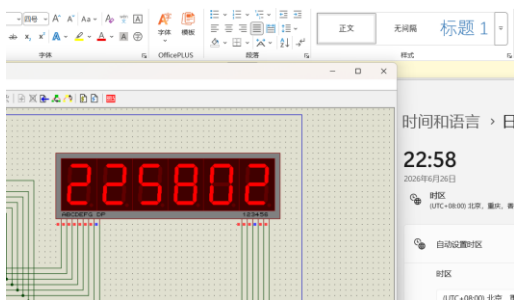
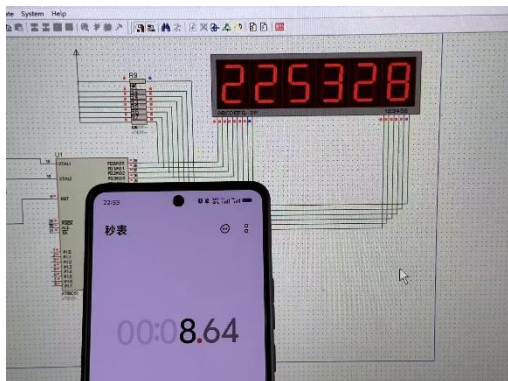
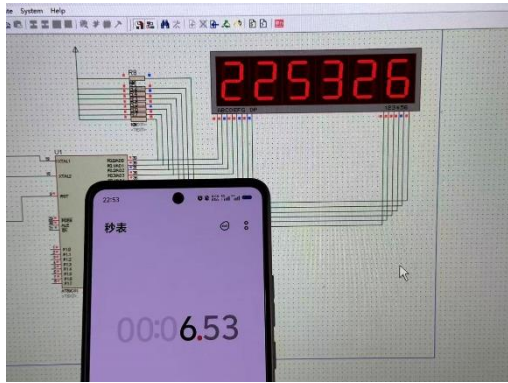


5、仿真图



现象：





6、表格分析：

显示时间	真实时间
22: 53: 00	22: 53: 00
22: 56: 07	22: 56: 08
22: 58: 21	22: 58: 21
22: 59: 11	22: 59: 11

分析：在运行过程因为进入中断或者循环判断的原因，偶尔由误差但是控制在一秒以内。

7、总结：

完成了数字钟的设计，并且增强了发现误差出现的原因的能力，增强了代码编写的能力。

总结：

本次单片机课程设计让我收获良多。从硬件电路搭建到程序编写调试，我熟悉了 IO 端口、定时器与数码管驱动原理。实操过程中多次遇到显示错乱、程序卡死等问题，通过逐段排查代码、检查线路接触，逐步找到故障原因。这次实践锻炼了我的软硬件协同调试能力，明白了理论知识必须结合动手实操。同时我也意识到自身经验不足，后续会多积累项目案例，提升嵌入式开发水平。